

Task 2 – Part C: Deriving the Attention Gradient by Hand

Setup (shapes as in the handbook)

```
-----
Q, K : T x d_k      (queries, keys)
V    : T x d_v      (values)
S = Q K^T / sqrt(d_k)  shape T x T  (scaled scores)
P = softmax(S)         shape T x T  (row-wise softmax; each row sums to 1)
A = P V               shape T x d_v (attention output)
```

We use index notation. i, a, c index query positions (rows of S/P); j, b, t index key positions (columns) or value features as noted. δ is the Kronecker delta. We write dL/dX for the gradient of a downstream scalar loss L with respect to X ; "the Jacobian" means the local derivative of one tensor w.r.t. another.

Part 1 – Warmup: dA/dV

A is linear in V given P , so this is the easy one.

$$A_{\{i\ j\}} = \sum_t P_{\{i\ t\}} V_{\{t\ j\}}$$

Differentiate w.r.t. a single entry $V_{\{k\ l\}}$:

$$\begin{aligned} dA_{\{i\ j\}} / dV_{\{k\ l\}} &= \sum_t P_{\{i\ t\}} * d(V_{\{t\ j\}})/d(V_{\{k\ l\}}) \\ &= \sum_t P_{\{i\ t\}} * \delta_{\{t\ k\}} * \delta_{\{j\ l\}} \\ &= P_{\{i\ k\}} * \delta_{\{j\ l\}} \end{aligned}$$

Interpretation: output entry $A_{\{i\ j\}}$ depends on $V_{\{k\ l\}}$ only when the feature matches ($j = l$), and the strength is the attention weight $P_{\{i\ k\}}$.

Backprop form (what code actually uses). Given the upstream gradient dL/dA ,

$$dL/dV = P^T (dL/dA) \quad (\text{shape } T \times d_v)$$

because each value row V_t feeds every output row A_i with weight $P_{\{i\ t\}}$, so its gradient is the P -weighted sum of the output-row gradients.

Part 2 – The softmax Jacobian

Let $p = \text{softmax}(s)$ for a single row, i.e.

$$p_i = \exp(s_i) / Z, \quad \text{where } Z = \sum_k \exp(s_k).$$

Differentiate p_i w.r.t. s_j . Use $d(\exp(s_i))/ds_j = \delta_{\{i\ j\}} \exp(s_i)$ and $dZ/ds_j = \exp(s_j)$.

$$\begin{aligned}
dp_i/ds_j &= [\delta_{ij} \exp(s_i) * Z - \exp(s_i) * \exp(s_j)] / Z^2 \\
&= \delta_{ij} * (\exp(s_i)/Z) - (\exp(s_i)/Z)(\exp(s_j)/Z) \\
&= \delta_{ij} p_i - p_i p_j \\
&= p_i (\delta_{ij} - p_j) \qquad \qquad \qquad [\text{QED}]
\end{aligned}$$

In matrix form, the Jacobian of one softmax row is

$$J = \text{diag}(p) - p p^T \qquad (\text{symmetric, singular: } J * 1 = 0).$$

Part 3 – Main result: dA/dQ

Chain rule: $dA/dQ = (dA/dP)(dP/dS)(dS/dQ)$.

We assemble the three local Jacobians, then give the compact backprop result.

(a) dS/dQ . $S_{\{a\}b} = (1/\sqrt{d_k}) \sum_d Q_{\{a\}d} K_{\{b\}d}$.

$$dS_{\{a\}b} / dQ_{\{c\}d} = (1/\sqrt{d_k}) * \delta_{\{a\}c} * K_{\{b\}d}$$

So $S_{\{a\}b}$ depends on row a of Q only ($\delta_{\{a\}c}$), and linearly in K .

(b) dP/dS . Softmax is applied independently per row, so cross-row terms vanish. Within row a (Part 2 with $s = S_{\{a\},:}$, $p = P_{\{a\},:}$):

$$\begin{aligned}
dP_{\{a\}b} / dS_{\{a\}c} &= P_{\{a\}b} (\delta_{\{b\}c} - P_{\{a\}c}) \\
dP_{\{a\}b} / dS_{\{a'\}c} &= 0 \qquad \qquad \qquad \text{if } a' \neq a
\end{aligned}$$

(c) dA/dP . From Part 1 ($A = P V$):

$$dA_{\{i\}j} / dP_{\{a\}b} = \delta_{\{i\}a} V_{\{b\}j}$$

Composing (backprop form). Start from upstream dL/dA and push backward:

$$\begin{aligned}
\text{Step 1: } dL/dP &= (dL/dA) V^T \qquad \qquad \qquad (T \times T) \\
[\text{ from (c): } dL/dP_{\{a\}b} &= \sum_j dL/dA_{\{a\}j} V_{\{b\}j}]
\end{aligned}$$

Step 2: go through the softmax row Jacobian (b). For each row a ,
 $dL/dS_{\{a\},:} = J_a^T (dL/dP_{\{a\},:})$ with $J_a = \text{diag}(P_a) - P_a P_a^T$:

$$dL/dS_{\{a\}b} = P_{\{a\}b} (dL/dP_{\{a\}b} - \sum_c dL/dP_{\{a\}c} P_{\{a\}c})$$

Compactly, per row (let $(*)$ denote elementwise product):
 $dS = P (*) (dP - \text{rowsum}(dP (*) P))$,
 where rowsum broadcasts the per-row scalar back over the row.

Step 3: through the scaled linear map (a):

$$\begin{aligned}
dL/dQ &= (1/\sqrt{d_k}) (dL/dS) K \qquad \qquad \qquad (T \times d_k) \\
[\text{ since } dL/dQ_{\{c\}d} &= (1/\sqrt{d_k}) \sum_b dL/dS_{\{c\}b} K_{\{b\}d}]
\end{aligned}$$

So the full path is:

$$dL/dA \xrightarrow{-V^T} dL/dP \xrightarrow{-\text{softmax } J} dL/dS \xrightarrow{-K/\sqrt{d_k}} dL/dQ$$

(For completeness, $dL/dK = (1/\sqrt{d_k}) (dL/dS)^T Q$, by the symmetric argument, and $dL/dV = P^T dL/dA$ from Part 1.)

Part 4 – Interpretation: why large logits kill the gradient

The gradient must pass through the softmax Jacobian $J = \text{diag}(P) - P P^T$ (Step 2 above). Look at what happens to J when the input logits S are large in magnitude.

When the scores in a row are large and spread out, softmax saturates: one entry $P_{\{a\ b^*\}} \rightarrow 1$ and the rest $\rightarrow 0$. Then every term of J vanishes:

- diagonal: $P_{\{a\ b^*\}}(1 - P_{\{a\ b^*\}}) \rightarrow 1*(1-1) = 0$ or $0*(1-0) = 0$,
- off-diagonal: $-P_{\{a\ b^*\}} P_{\{a\ c\}} \rightarrow 0$ (a product of a ~ 1 and a ~ 0 , or two ~ 0 s).

So $J \rightarrow 0$, and by Step 2 $dL/dS \rightarrow 0$. The gradient that reaches Q and K is therefore tiny: learning stalls (this is the softmax analogue of a saturated sigmoid).

Now connect to $\sqrt{d_k}$. If the entries of Q and K are roughly independent with mean 0 and variance 1, then each dot product $q \cdot k = \sum_{d=1..d_k} q_d k_d$ has variance about d_k (a sum of d_k unit-variance terms), i.e. standard deviation $\sqrt{d_k}$. Without scaling, the score magnitudes grow like $\sqrt{d_k}$, pushing softmax into exactly the saturated regime above as d_k grows. Dividing by $\sqrt{d_k}$ normalizes the variance back to ~ 1 regardless of head dimension, so softmax stays in its responsive, non-saturated region and J (hence the gradient) stays healthy. This is the formal version of the Task 1 answer: the $\sqrt{d_k}$ factor exists to keep the softmax gradient from vanishing.